



泛型类与泛型方法的编写

北京理工大学计算机学院
金旭亮



1 编写泛型类

泛型类示例：Pair<T>

定义了泛型参数的类成为泛型类，泛型参数可用于定义字段类型、方法参数类型和返回值类型。

```
//泛型参数为T
public class Pair<T> {
    // (1) 用T定义字段类型
    private T first;
    private T second;
    // (2) 用T定义构造方法的参数类型
    public Pair(T first, T second) {
        this.first = first; this.second = second; }
    // (3) 用T定义函数返回值类型
    public T getFirst() { return first; }
    // (4) 用T定义方法参数类型
    public void setFirst(T newValue) { first = newValue; }
    .....
}
```

泛型类的“实例化”

前面定义了泛型类Pair<T>，但在使用时，必须给T指定一个具体的类型（比如String）。“Pair<String>”就是“Pair<T>”的“**实例化**（**instantiate**）”。

示例：Pair.java

```
public static Pair<String> minmax(String[] a) {  
    if (a == null || a.length == 0) {  
        return null;  
    }  
    String min = a[0];  
    String max = a[0];  
    for (int i = 1; i < a.length; i++) {  
        if (min.compareTo(a[i]) > 0) {  
            min = a[i];  
        }  
        if (max.compareTo(a[i]) < 0) {  
            max = a[i];  
        }  
    }  
    return new Pair<String>(min, max);  
}
```

泛型使用“须知” (1)



泛型参数必须是引用类型，不适用于基本数据类型，因此，`Pair<double>`是错的，只能是`Pair<Double>`。



泛型参数类型变量可以引用一个“原始类型 (raw type)”的对象（反过来也可以），但编译时会产生警告，实际开发中不推荐使用这个特性：

```
ArrayList<String> strs = new ArrayList();  
ArrayList strs2 = new ArrayList<String>();
```

泛型使用“须知” (2)

- ✓ 不能定义泛型化数组

```
Pair<String>[] table = new Pair<String>[10];
```



- ✓ 不能直接创建泛型类型的实例

```
public class Pair<T> {  
    ...  
    public Pair() {  
        first = new T();  
        second = new T();  
    }  
    ...  
}
```



泛型使用“须知” (3)



泛型类型不能直接或间接继承自Throwable。

```
public class Problem<T> extends Exception
```



我们无法抛出或捕获泛型类型的异常对象

```
catch (T e) {...}
```



泛型使用“须知” (4)



不能定义静态泛型成员，以下代码将无法通过编译：

```
class MyClass<T> {  
    public static T value;  
    public static T f() {    }  
}
```



从泛型类派生子类 (1)



从泛型基类派生子类时，应该给其指定一个具体的类型

```
class MyChild extends MyClass<T> {  
}
```



```
class MyChild extends MyClass<String> {  
}
```



从泛型类派生子类 (2)



如果MyClass是泛型类，在定义子类时不指定泛型参数，则 MyClass 的泛型参数默认为 **Object**。

```
class MyChild extends MyClass {  
}
```



```
class MyChild extends MyClass<Object> {  
}
```



除了定义一个完整的泛型类，我们还可以定义“轻量级”的泛型方法

2 编写泛型方法

定义泛型方法

1 可以在普通类或泛型类中定义泛型方法：

```
class ArrayAlg {  
    public static <T> T getMiddle(T[] a) {  
        return a[a.length / 2];  
    }  
}
```

2 使用泛型方法：

```
String[] names = ... ;  
String middle = ArrayAlg.<String>getMiddle(names);
```

上下两句代码是等价的



编译器能“自动推断”

```
String middle = ArrayAlg.getMiddle(names);
```

泛型方法类型的“自动推断”示例



定义：

```
//声明一个泛型方法，该泛型方法中带一个T形参，
static <T> void fromArrayToCollection(T[] arr,
                                     Collection<T> collection) {

    //在调用JDK所定义的泛型方法时无需指明类型参数，
    //编译器会进行类型推断，以确定具体的数据类型。
    Collections.addAll(collection, arr);
}
```



使用：

```
String[] stringArray = new String[100];
Collection<String> stringCollection = new ArrayList<>();
//泛型方法中的参数T，被推断为String类型
fromArrayToCollection(stringArray, stringCollection);
```

数目可变的泛型方法参数



泛型方法支持使用 “...” 定义个数可变的参数：

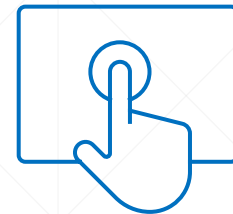
```
class Lists {  
    public static <T> List<T> toList( T... arr ) {  
        List<T> list = new ArrayList<T>();  
        for (T elt : arr) list.add(elt);  
        return list;  
    }  
}
```



使用示例：

```
List<Integer> ints = Lists.toList ( 1, 2, 3 );  
List<String> words = Lists.toList ( "hello" , "world" );
```

泛型参数的命名规范



当你设计自己的泛型类或泛型方法时，最好遵循以下惯例命名泛型参数：

- T: Type的简写，用于定义单独一个泛型参数，例如：<T>
- S: Second的简写，用于定义第二个泛型参数，例如：<T,S>
- E: Element的简写，用于定义泛型集合，例如：<E>
- K: Key的简写，V: Value的简写，通常用于定义Map集合，例如，<K,V>

下一讲



泛型多态与泛型约束